

MEDIATEK

MT8668 Hypervisor UART User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	李文中	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 UART 4

1.1 概述 4

1.1.1 简单介绍 4

1.1.2 名词解释 4

1.2 架构/流程概述 4

1.2.1 UART 介绍 4

1.2.2 MT8668 UART 特征 5

1.2.3 Uart 8250 框架传输 flow 6

1.3 配置/客制化指南 6

1.3.1 Linux 构建配置 6

1.3.2 添加 UART DTS 节点 7

1.3.3 添加 APDMA DTS 节点 7

1.3.4 添加 GPIO 设定 8

1.3.5 测试和调试 9

1.4 常见问题/故障排查 9

1.4.1 UART 无法输入输出 9

1.4.2 UART 乱码问题 9

1.4.3 UART 不打印内核日志 10

附件一 附加条款 11

图片目录

图 1-1. SoC UART 和设备 UART 之间的引脚连接 4

表格目录

表 1-1. 名词解释 4

1 UART

1.1 概述

1.1.1 简单介绍

本章节介绍 MT8668 UART 控制器的硬件、软件及其功能。

1.1.2 名词解释

表 1-1. 名词解释

缩写	解释
CTS	Clear To Send 发送准备
DMA	Direct Memory Access 直接内存访问
FIFO	First In First Out 先入先出
RTS	Request To Send 接收准备
RX	Receiver 接收端口
TX	Transmitter 发送端口
UART	Universal Asynchronous Receiver/Transmitter 通用异步收发器

1.2 架构/流程概述

1.2.1 UART 介绍



图 1-1. SoC UART 和设备 UART 之间的引脚连接

UART 是一种串行通信接口协议，常用于将计算机或者微控制器与外围设备进行通信。TX、RX、CTS 和 RTS 是 UART 协议中的四个重要信号，它们分别用于传输数据和控制传输流程。

TX (Transmit): 发送端口

TX 为 UART 中的发送端口，即计算机或微控制器用于发送数据的端口。TX 把待发送的数据逐位地转化为串行数据帧，并通过序列化后的数据电平在通信线上向外发送。当发送完成，TX 状态将转为低电平。

RX (Receive): 接收端口

RX 为 UART 中的接收端口，即计算机或微控制器用于接收数据的端口。RX 将发送端发出的数据经过反序列化后转化为待接收的数据帧。当数据接收完成后，RX 状态将转为高电平。

CTS (Clear To Send): 发送准备

CTS 是一个输出口，它向发送方传递一种状态信息，用来指示 RX 端是否已经准备好接收新的数据。当且仅当 CTS 状态为高电平时，表示 RX 端已准备好接收数据；当 CTS 状态为低电平时，表示 RX 端正在处理数据，此时发送方应该暂停传输。

RTS (Ready To Send): 接收准备

RTS 是一个输入口，它向接收方传递一种状态信息，用来指示 TX 端是否已经准备好发送新的数据。当 TX 端已经准备好发送数据时，RTS 状态为低电平；当 TX 端不准备发送数据时，RTS 状态为高电平，此时接收方应该暂停接收。

CTS 和 RTS 的作用是在数据传输前进行握手协商以避免发送方和接收方竞争同一个时刻来进行数据传输，从而保证数据传输时的可靠性和稳定性。

综上所述，UART 的 TX、RX、CTS 和 RTS 四个信号是 UART 协议中的核心信号，配合使用可以实现数据的串行传输及有效的流控管理。

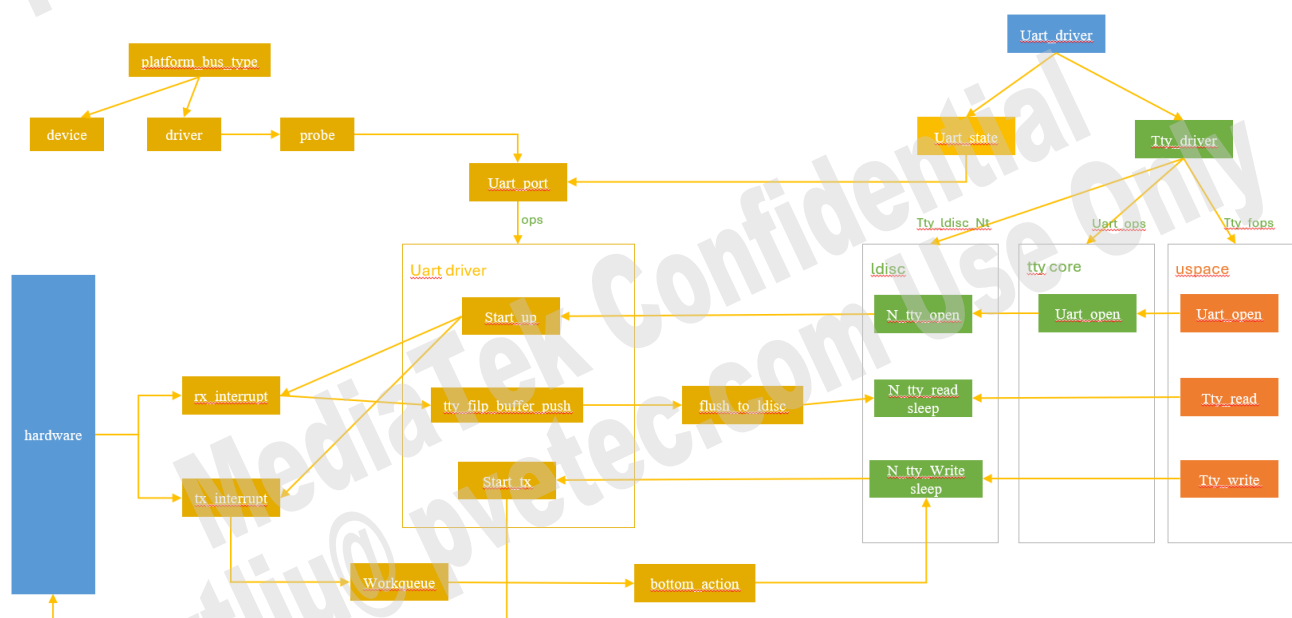
图 1-1 展示的是 SoC 与外围设备通过 UART 进行数据传输的硬件连接，其中 RTS 和 CTS 用于硬件流控管理，可以根据使用场景决定移除或保留。

1.2.2 MT8668 UART 特征

- 提供 3 路串口
- UART0/UART1 是 2 针 (TX、RX) UART 通道
- UART2 是一个 4 针 (TX、RX、CTS、RTS) UART 通道
- 支持 M16C450 和 M16550A 操作模式
- 兼容标准软件驱动程序
- 传输系统：异步

- 数据长度：5 到 8 位
- 硬件流控：基于 CTS/RTS 的自动收发控制
- 软件流控：使用特殊字符 Xon/Xoff 进行软件流控
- 波特率可编程为 300 bps 至 3 Mbps
- 中断请求：接收中断/发送中断
- 数据传输：支持 DMA（发送/接收）传输

1.2.3 Uart 8250 框架传输 flow



该架构图描绘了 UART 子系统中的双向数据流转机制。在发送路径 TX 上，用户数据经由 TTY 核心及线路规程 N_TTY 处理后，通过驱动层的 start_tx 接口写入硬件 FIFO，最终触发发送中断完成传输。在接收路径 RX 上，物理硬件收到数据后触发接收中断，驱动层通过中断服务程序（ISR）读取数据，并利用 Workqueue 等下半部机制（Bottom Half）将数据通过 tty_flip_buffer_push 推送至线路规程缓冲区，最终供用户空间读取。整个过程由 TTY 层负责流控与缓冲，UART 驱动层负责硬件操作。

1.3 配置/客制化指南

1.3.1 Linux 构建配置

配置项目：

- 分配在 Linux 内核下支持的最大 UART（通用异步收发器/发射器）端口数量

```
CONFIG_SERIAL_8250_NR_UARTS=3
CONFIG_SERIAL_8250_RUNTIME_UARTS=3
```

- 启用 UART DMA 支持

```
CONFIG_DMA_MTK_UART=y
```

- 启用对 8250 UART 协议 和 MTK UART 驱动的支持

```
CONFIG_SERIAL_8250=y
```

1.3.2 添加 UART DTS 节点

```
uart0: serial@11001000 {
    compatible = "mediatek,mt6577-uart";
    reg = <0 0x11001000 0 0x1000>;
    interrupts = <GIC_SPI 522 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&clk26m>, <&pericfg_ao_reg_clk
    CLK_PERICFG_AO_REG_PERI_UART0_UART>;
    clock-names = "baud", "bus";
    dmas = <&apdma 0 &apdma 1>;
    dma-names = "tx", "rx";
    uart-line = <0>;
};
```

1.3.3 添加 APDMA DTS 节点

Device node of APDMA:

```
apdma: dma-controller@11300c00 {
    compatible = "mediatek,mt6985-uart-dma";
    reg = <0 0x11300c00 0 0x80>,
        <0 0x11300c80 0 0x80>,
        <0 0x11300d00 0 0x80>,
        <0 0x11300d80 0 0x80>,
        <0 0x11300e00 0 0x80>,
        <0 0x11300e80 0 0x80>;
    interrupts = <GIC_SPI 560 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 561 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 562 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 563 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 564 IRQ_TYPE_LEVEL_HIGH 0>,
        <GIC_SPI 565 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&pericfg_ao_reg_clk
    CLK_PERICFG_AO_REG_PERI_DMA_B_UART>;
    clock-names = "apdma";
    dma-requests = <6>;
    #dma-cells = <1>;
};
```

```
};
```

1.3.4 添加 GPIO 设定

- 配置 GPIO 并设置为开，默认为开:

```
&uart0 {
    pinctrl-names = "default", "sleep ";
    pinctrl-0 = <&uart0_pin_default>;
    pinctrl-1 = <&uart0_pin_sleep>;
    status = "okay";
};
```

- 如在 OS 中未使用，请将状态设置为关:

```
&uart0 {
    pinctrl-names = "default", "sleep ";
    pinctrl-0 = <&uart0_pin_default>;
    pinctrl-1 = <&uart0_pin_sleep>;
    status = "disabled";
};
```

- 配置 PIO 节点中与 UART 通信相关的默认/休眠引脚:

```
&pio {
    uart0_pin_default: uart0_pin_uart_mode {
        pins_rx {
            pinmux = <PINMUX_GPIOxxx_FUNC_URXD0>;
            input-enable;
            bias-pull-up;
        };
        pins_tx {
            pinmux = <PINMUX_GPIOxxx_FUNC_UTXD0>;
            output-high;
        };
    };

    uart0_pin_sleep: uart0_pin_gpio_mode {
        pins_rx {
            pinmux = <PINMUX_GPIOxxx_FUNC_GPIOxxx>;
            bias-pull-down;
        };
        pins_tx {
            pinmux = <PINMUX_GPIOxxx_FUNC_GPIOxxx>;
            bias-pull-down;
        };
    };
};
```

1.3.5 测试和调试

1. 设置波特率 921600, 8 位数据位, 1 位停止位, 无奇偶校验, 如 UART1

```
stty -F /dev/ttyS1 ispeed 921600 ospeed 921600 cs8
```

2. 获取当前 UART 波特率以及数据位配置信息

```
stty -F /dev/ttyS1 -a
```

3. UART 数据获取, 可以使用以下命令接收 UART1 数据并通过 console 打印出来

```
cat /dev/ttyS1
```

4. UART 发送数据, 可以使用以下命令让 UART1 发送字符串"123"

```
echo 123 > /dev/ttyS1
```

1.4 常见问题/故障排查

1.4.1 UART 无法输入输出

1. 请按照前文的提供的信息, 检查 dts 配置是否正确
2. 请使用 Linux 命令检查 UART 引脚模式是否切换至 UART 模式

内核里查看 PIN 的状态, 输入以下命令:

```
# cat /proc/mtk_gpio/soc.pinctrl
# cd /proc/mtk_gpio/
# cat soc.pinctrl
```

e.g.,

```
# cat /proc/mtk_gpio/soc.pinctrl
```

```
PIN: [MODE] [DIR] [DOUT] [DIN] [PULL_EN] [PULL_SEL] [IES] [SMT] [DRIVE] ( [R1] [R0] )
0: 0 0 0 0 1 0 1 0 0
1: 0 0 0 0 1 0 1 0 0
2: 0 1 1 1 1 0 1 0 0
3: 6 0 0 0 1 0 1 0 0
```

1.4.2 UART 乱码问题

1. 请按照前文提供的信息检查波特率是否一致
2. 请检查 UART source_clk 是否符合预期, 通常 source_clk 会设置成 26 MHz.

1.4.3 UART 不打印内核日志

User 版本中，在默认情况下，image 进入 shell 之后便不再打印内核日志，而 engine/user-debug image 则默认继续打印内核日志。可以参考以下方法进行修改：

- echo 1 > /proc/mtprint：用于在 adb shell 可用后启用 UART 日志记录
- setprop persist.uartconsole.enable 1：用于在启动后启用 UART 日志记录
- 通过修改代码为用户加载默认启用 UART 日志记录

```
vendor/mediatek/proprietary/bootable/bootloader/lk/app/mt_boot/mt_boot.c  
cmdline_append("printk.disable_uart=0"); // BUILD_TYPE_USER
```

```
device/mediatek/mt6881/init.mt6881.rc mask "write /proc/bootprof 0"  
# write /proc/bootprof 0
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。